MuJoCo CPU vs. MuJoCo Warp Derivative Validation Plan

Abstract

This document describes how CPU MuJoCo computes first-order discrete-time dynamics derivatives, the numerical concerns that arise when reproducing the same calculation in MuJoCo Warp (MJWarp), and a practical validation plan for testing GPU-batched finite differences against an existing CPU MuJoCo derivative pipeline. The central goal is to determine whether MJWarp can provide sufficiently accurate, stable, and useful transition Jacobians for contact-rich trajectory optimization without relying on automatic differentiation.

Core question. Can CPU finite-difference dynamics Jacobians be replaced by GPU-batched finite differences without losing derivative quality around the contact-rich states that matter for trajectory optimization?

Recommended first milestone. Do not begin with an optimizer. First reproduce `mjd_transitionFD` with a simple CPU black-box central-difference wrapper, then port that exact perturbation and state-difference logic to MJWarp and perform an epsilon sweep. This isolates implementation errors from GPU precision and simulator differences.

1 Executive Summary

CPU MuJoCo does not compute the discrete-time transition Jacobians by solving a separate sensitivity optimization problem. The public routine `mjd_transitionFD` computes finite-difference approximations of the one-step simulator map and can use forward or centered differences [1].

For a discrete-time simulator

$$x_{k+1} = F(x_k, u_k), \tag{1}$$

the desired Jacobians are

$$A = \frac{\partial F}{\partial x}, \qquad B = \frac{\partial F}{\partial u}. \tag{2}$$

The implementation is more careful than a naive black-box finite-difference loop. It uses staged simulator calls to avoid recomputing unaffected portions of the dynamics pipeline, handles control limits, perturbs generalized positions in the n_v-dimensional configuration tangent space, and differences quaternion-containing states with manifold-aware MuJoCo routines [2].

MJWarp is attractive because the perturbed simulations required by finite differences are independent and can be evaluated as a large GPU batch. The primary numerical risk is that MJWarp simulation arrays use single precision, whereas standard CPU MuJoCo uses double precision by default. MJWarp documentation also notes numerical differences related to floating-point representation and GPU nondeterminism [3, 4].

2 What CPU MuJoCo Actually Does

2.1 Discrete-Time Finite Differences

For the top-level step map, MuJoCo treats the current state and control as inputs and the next state as the output. With centered differences, a state column is approximated as

$$A_{:,i} \approx \frac{F(x \oplus he_i, u) - F(x \oplus (-he_i), u)}{2h}, \tag{3}$$

and a control column as

$$B_{:,j} \approx \frac{F(x, u + he_j) - F(x, u - he_j)}{2h}. \tag{4}$$

Here $\oplus$ emphasizes that generalized-position perturbations must respect the configuration manifold rather than directly adding to quaternion components.

There is no additional mathematical program whose solution is differentiated to obtain A and B. Each evaluation of F, however, still runs the MuJoCo simulation pipeline, including collision detection, constraint construction, the selected contact/constraint solver, actuation, and integration. Therefore the Jacobian is a finite difference of the complete simulator response.

2.2 Dimensions and Tangent-Space State

For a model with generalized velocity dimension n_v, actuator-state dimension n_a, and control dimension n_u, MuJoCo returns

$$A \in \mathbb{R}^{(2n_v+n_a)\times(2n_v+n_a)}, \qquad B \in \mathbb{R}^{(2n_v+n_a)\times n_u}. \tag{5}$$

This matters for floating-base robots because quaternion coordinates increase n_q without increasing the tangent-space dimension. The derivative state is represented as

$$\delta x = \begin{bmatrix} \delta q_{\tan} \\ \delta v \\ \delta a \end{bmatrix} \in \mathbb{R}^{2n_v+n_a}. \tag{6}$$

In the CPU finite-difference implementation, position perturbations are constructed as n_v-dimensional tangent vectors and applied with `mj_integratePos`. Output position differences use `mj_differentiatePos` when required [2]. A custom MJWarp derivative implementation should reproduce these semantics.

2.3 Efficiency Tricks in `mjd_transitionFD`

MuJoCo exploits the staged simulation pipeline instead of blindly rerunning every calculation from scratch for every perturbation [2].

- **Controls:** perturb `ctrl` and call a skipped-step path beginning at the velocity stage where possible.
- **Activations:** similarly reuse position- and velocity-dependent computations when possible.
- **Velocities:** perturb `qvel` and skip position-only work.
- **Positions:** perturb through `mj_integratePos`; position-dependent quantities must be recomputed.

- **Control limits:** if a centered perturbation would violate an actuator control limit, a one-sided perturbation may be used.
- **Warm start:** the routine preserves the warm-start acceleration state so repeated pipeline calls are more reproducible.
- **Solver settings:** MuJoCo documentation recommends controlling solver convergence carefully during derivative computation; fixed iteration behavior can improve repeatability [1].

Implication for the GPU prototype. A first MJWarp prototype does not need to reproduce CPU stage-skipping optimizations. It can run full batched simulation steps for every perturbation. Correctness should come first; GPU batching may already provide enough throughput. Stage-specific optimizations can be added later only if profiling justifies them.

3 Why GPU Finite Differences Are Attractive

For centered differences, each Jacobian coordinate requires two independent simulator evaluations. If the tangent-state dimension is n_x and the control dimension is n_u, then

$$N_{\text{world}} = 2(n_x + n_u) \tag{7}$$

perturbed worlds are required, plus an optional nominal world.

This workload is embarrassingly parallel. A GPU simulator designed for high-throughput batched worlds can evaluate all $+h$ and $-h$ perturbations simultaneously.

3.1 Directly Differentiate the Complete Shooting Map

Suppose one shooting interval contains S simulator substeps while the control is held fixed. Define the interval map

$$x_{k+1} = F_H(x_k, u_k) = f^S(x_k, u_k). \tag{8}$$

Instead of forming substep Jacobians and chaining them, the GPU experiment can finite-difference F_H directly:

$$A_H[:, i] \approx \frac{F_H(x \oplus he_i, u) - F_H(x \oplus (-he_i), u)}{2h}, \tag{9}$$

$$B_H[:, j] \approx \frac{F_H(x, u + he_j) - F_H(x, u - he_j)}{2h}. \tag{10}$$

This is particularly attractive for multiple-shooting trajectory optimization because each GPU world can execute the full S-substep shooting interval independently. The direct interval derivative should be attempted only after the one-step implementation is validated.

4 Main GPU and MJWarp Concerns

4.1 Float32 Cancellation in Finite Differences

MJWarp uses single-precision floating point for simulation arrays, while standard CPU MuJoCo uses double precision by default [3, 4, 6]. Approximate machine epsilon values are

$$\epsilon^{32}_{\text{mach}} \approx 1.19 \times 10^{-7}, \qquad \epsilon^{64}_{\text{mach}} \approx 2.22 \times 10^{-16}. \tag{11}$$

A centered finite difference subtracts two nearby simulator outputs. If h is too small, the numerator can be dominated by float32 rounding, solver noise, or nondeterminism. If h is too large, truncation error and simulator nonlinearity dominate.

The practical question is therefore not whether float32 is intrinsically unusable. It is whether there exists a useful interval

$$h_{\min} < h < h_{\max} \tag{12}$$

for which the derivative is both numerically resolvable and sufficiently local.

Do not assume the CPU epsilon will transfer. An epsilon that works in double precision, such as 10^{-6}, may be too small in float32. The test should sweep h over several orders of magnitude instead of fixing one value in advance.

4.2 Contact-Set Sensitivity

The most important failure mode is not a small arithmetic perturbation. It is a numerical perturbation that changes the discrete contact or constraint set.

A recent MJWarp issue gives a concrete near-degenerate mesh example in which float32 support-point rounding changed contact behavior after a common translation, despite unchanged relative geometry [5]. The example is not evidence that ordinary humanoid contact is generally unreliable, but it illustrates why contact diagnostics are necessary.

For finite differences, the two states

$$x \oplus he_i, \qquad x \oplus (-he_i) \tag{13}$$

may produce different contact pairs or active constraints. A large derivative column may then result from

1. a genuine nonsmooth contact transition,
2. a difference between CPU MuJoCo and MJWarp physics/collision handling, or
3. float32-induced mode switching.

These cases should not be lumped together.

4.3 GPU Nondeterminism

MJWarp documentation notes that GPU execution is not deterministic and that small run-to-run numerical differences can occur because of ordering effects such as atomic operations [3]. Finite differences can amplify output noise. If simulator output noise has magnitude η, then derivative noise can scale approximately as

$$O\left(\frac{\eta}{h}\right). \tag{14}$$

Therefore selected test points should be repeated several times with identical inputs to measure Jacobian variability.

4.4 Solver Convergence and Early Termination

Finite-difference comparisons are meaningful only when perturbed simulations use comparable solver settings. On CPU, MuJoCo recommends controlling solver convergence carefully during derivative computation; forcing a fixed amount of solver work can improve repeatability [1]. The same principle should be followed in MJWarp as closely as its available options permit.

4.5 CPU Single Precision as an Optional Control

MuJoCo uses double precision by default, but the codebase supports a single-precision build through `mjUSESINGLE` [6]. If practical, a CPU-float32 build is a useful control experiment:

$$\text{CPU64} \quad \text{vs.} \quad \text{CPU32} \quad \text{vs.} \quad \text{MJWarp32}. \tag{15}$$

This helps separate precision-induced error from simulator-implementation differences. It is optional for the first prototype.

5 Test Plan

5.1 Phase 0: Freeze a Reproducible Configuration

1. Use the same MJCF/model parameters, timestep, integrator, solver type, friction cone, actuator settings, gravity, contact parameters, and initial state for CPU and MJWarp.
2. Disable sources of randomness where possible.
3. Record MuJoCo, MJWarp, Warp, CUDA, GPU, compiler/build, and driver versions.
4. Save a small set of representative trajectory nodes as fixtures: `qpos`, `qvel`, actuator state if used, `ctrl`, and any other required simulator state.
5. Keep derivative solver iteration/tolerance settings fixed and log them with every result.

5.2 Phase 1: Nominal Transition Parity

Before comparing derivatives, compare the unperturbed simulator map:

$$e_F = d_x(F_{\text{Warp}}(x, u), F_{\text{CPU}}(x, u)) \,, \tag{16}$$

where d_x is a manifold-aware state distance.

Test one simulator step first, then the full S-substep shooting interval. Log at least

- generalized-position tangent error,
- generalized-velocity error,
- actuator-state error,
- contact count,
- contact geometry/body pairs,
- contact distances or gaps,
- active constraint count.

If nominal rollouts diverge materially, derivative mismatch is expected; nominal parity should be resolved or characterized first.

5.3 Phase 2: Reimplement Finite Differences on CPU

Write a simple black-box CPU central-difference routine using the same tangent-space perturbations and state differencing planned for MJWarp. Compare it against `mjd_transitionFD` using the same h.

```
for i in range(nx):
    x_plus = tangent_perturb(x, +h * e_i)
    x_minus = tangent_perturb(x, -h * e_i)
    y_plus = cpu_rollout(x_plus, u)
    y_minus = cpu_rollout(x_minus, u)
    A[:, i] = tangent_state_difference(y_minus, y_plus) / (2*h)

for j in range(nu):
    y_plus = cpu_rollout(x, u + h * e_j)
    y_minus = cpu_rollout(x, u - h * e_j)
    B[:, j] = tangent_state_difference(y_minus, y_plus) / (2*h)
```

This catches common implementation errors before introducing GPU differences:

- quaternion perturbation,
- quaternion output differencing,
- state packing,
- actuator-state handling,
- control indexing,
- Jacobian transpose/layout conventions.

5.4 Phase 3: Implement the Same Perturbation Batch in MJWarp

1. Allocate one world for every $+h$ and $-h$ state perturbation and every $+h$ and $-h$ control perturbation.
2. Copy the same nominal state and control into all worlds.
3. Apply exactly one perturbation per world.
4. Use tangent-space generalized-position perturbations; do not directly perturb raw quaternion components.
5. Run exactly one MJWarp step for the one-step test.
6. Gather final states, perform manifold-aware output differencing, and assemble A_{Warp} and B_{Warp}.
7. Repeat the same pattern with S steps per world to obtain the direct shooting-interval Jacobian.

5.5 Phase 4: Epsilon Sweep

Use a logarithmic sweep instead of a single value. A reasonable initial sweep is

$$h \in \left\{10^{-2},\ 3\times10^{-3},\ 10^{-3},\ 3\times10^{-4},\ 10^{-4},\ 3\times10^{-5},\ 10^{-5},\ 3\times10^{-6},\ 10^{-6}\right\}. \tag{17}$$

For each h and each state fixture, compute

$$E_A(h) = \frac{\|A_{\text{Warp}}(h) - A_{\text{CPU64}}\|_F}{\max(\|A_{\text{CPU64}}\|_F, \varepsilon_{\text{den}})}, \tag{18}$$

$$E_B(h) = \frac{\|B_{\text{Warp}}(h) - B_{\text{CPU64}}\|_F}{\max(\|B_{\text{CPU64}}\|_F, \varepsilon_{\text{den}})}. \tag{19}$$

Also record

- per-column relative and absolute error,
- maximum elementwise error,
- 50th, 90th, and 99th percentile errors,
- cosine similarity of corresponding columns when the reference column norm is meaningful,
- repeated-run standard deviation for selected MJWarp h values.

The expected pattern is a truncation-versus-roundoff tradeoff: error may decrease as h becomes smaller, reach a useful minimum, and then increase once float32 noise and nondeterminism dominate.

5.6 Phase 5: Directional Taylor Test

Direct matrix agreement is useful, but the more important question is whether the Jacobian predicts local simulator changes.

Choose random tangent-space directions δx and δu and evaluate

$$r(\alpha) = d_x(F(x \oplus \alpha\delta x, u + \alpha\delta u), F(x,u) \oplus \alpha(A\delta x + B\delta u)). \tag{20}$$

Away from nonsmooth transitions, a valid first-order model should show a decreasing residual as α decreases, with approximately second-order behavior over a useful range before roundoff or simulation noise dominates.

Run this test separately for

- CPU Jacobians against CPU rollouts,
- MJWarp Jacobians against MJWarp rollouts,
- optionally MJWarp Jacobians against CPU rollouts.

5.7 Phase 6: Contact Diagnostics

For every nominal and perturbed rollout, log enough contact information to classify bad derivative columns:

- number of raw contacts,
- number of active constraints,
- geom/body pairs,
- signed contact distances or gaps,
- contact margins,
- normal and tangential contact forces when available,
- solver iteration count and termination/residual information when available,
- whether $+h$ and $-h$ perturbations preserve the nominal contact set.

Report derivative errors separately for contact-stable and contact-changing perturbations. Mixing both into one Frobenius norm can hide the actual behavior.

5.8 Phase 7: Representative State Suite

State category	**Why include it**	**Expected difficulty**
Flight / no contact	Baseline smooth rigid-body dynamics	Low
Stable stance	Contact active but mode should remain fixed	Moderate
Loaded foot contact	Tests solver and friction sensitivity	Moderate–high
Near contact onset	Perturbations may create/remove constraints	High / nonsmooth
Near contact release	Same mode-change issue in reverse	High / nonsmooth
Sliding / friction transition	Sensitive to friction cone and solver	High
Self-contact, if used	Complex collision/contact set	High
Highly dynamic impact	Large accelerations and contact changes	Very high

Table 1: Recommended representative states for derivative validation.

6 Suggested Output Files and Metrics

For each state fixture and epsilon, save a compact record that is easy to plot and regress later. Recommended fields include

- fixture ID and trajectory node/time,
- contact category,
- CPU and MJWarp nominal next-state errors,
- h, centered/forward flag, number of worlds, and number of simulation substeps S,

- A_{CPU}, B_{CPU}, A_{Warp}, and B_{Warp} or compressed summaries plus optional matrix dumps,
- Frobenius relative errors,
- per-column errors,
- maximum and percentile errors,
- column cosine similarities,
- repeated-run variability,
- contact-set equality flags,
- timing for setup/copy, GPU rollout, gather, Jacobian assembly, and CPU `mjd_transitionFD`.

6.1 Most Informative Plots

1. Relative A error versus h, with curves separated by state category.
2. Relative B error versus h.
3. Nominal CPU-vs.-MJWarp transition error versus state category.
4. Derivative error split by contact-stable and contact-changing perturbations.
5. Taylor-test residual versus α on log–log axes.
6. Runtime versus world count and versus shooting substeps S.

7 How to Decide Whether It Is Good Enough

Do not require bitwise or high-decimal agreement with CPU MuJoCo. The relevant standard is whether the derivative is stable, locally predictive, and useful to the optimizer.

A practical decision sequence is

1. **Smooth/no-contact states:** identify an h region where MJWarp Jacobians are stable across h and repeated runs and agree reasonably with CPU.
2. **Stable-contact states:** verify that a similar window exists and that contact sets remain unchanged across most perturbations.
3. **Transition states:** characterize separately; large or discontinuous derivatives may reflect genuine contact nonsmoothness rather than implementation failure.
4. **Shooting-interval test:** confirm that direct S-step GPU finite differences remain stable and do not accumulate excessive rollout divergence or numerical noise.
5. **Optimizer test:** run a small trajectory optimization with identical initialization and compare convergence, constraint violation, final objective, NLP iteration count, and total wall time.

Best outcome. A clear intermediate epsilon region exists—likely larger than the CPU double-precision epsilon—where MJWarp Jacobians are sufficiently repeatable, predict local rollouts well, and give optimizer behavior comparable to CPU while reducing derivative wall time.

Failure mode worth identifying early. If no epsilon yields stable derivatives even in ordinary stable-contact states, or if run-to-run noise and contact-set changes dominate the finite difference, then MJWarp finite differences are unlikely to be a drop-in replacement for the current first-order NLP pipeline.

8 Implementation Checklist

☐ Create saved CPU state/control fixtures from existing trajectories.

☐ Implement `tangent_perturb_q` using MuJoCo-compatible generalized-position integration.

☐ Implement `tangent_state_difference` using MuJoCo-compatible orientation differencing.

☐ Implement CPU black-box centered finite differences and verify against `mjd_transitionFD`.

☐ Implement one-step MJWarp batched perturbation worlds.

☐ Verify nominal CPU/MJWarp transition parity.

☐ Add epsilon sweep and matrix error metrics.

☐ Add repeated MJWarp runs to quantify nondeterminism.

☐ Add contact-pair and active-constraint logging.

☐ Add the Taylor directional-derivative test.

☐ Extend from one step to the full S-substep shooting interval.

☐ Benchmark derivative wall time on CPU versus GPU.

☐ Only after the above passes, test inside a small optimization problem.

9 Minimal GPU Batch Structure

The conceptual world layout is

$$[x{+}0,\ x{-}0,\ x{+}1,\ x{-}1, \ldots, u{+}0,\ u{-}0,\ u{+}1,\ u{-}1, \ldots]. \tag{21}$$

```
nx = 2*nv + na
nworld = 2*(nx + nu)

# World layout:
# [x+0, x-0, x+1, x-1, ..., u+0, u-0, u+1, u-1, ...]

initialize_all_worlds_from_nominal(x, u)

for i in range(nx):
    tangent_perturb_world(2*i, state_coord=i, amount=+h)
    tangent_perturb_world(2*i + 1, state_coord=i, amount=-h)

base = 2*nx
for j in range(nu):
```

```
    ctrl[base + 2*j, j] += h
    ctrl[base + 2*j + 1, j] -= h

for s in range(S):
    mjw.step(model, data)

Y = gather_final_states()

for i in range(nx):
    A[:, i] = state_diff(Y[2*i+1], Y[2*i]) / (2*h)

for j in range(nu):
    p = base + 2*j
    B[:, j] = state_diff(Y[p+1], Y[p]) / (2*h)
```

The pseudocode is conceptual. The generalized-position part of `tangent_perturb_world` and the output `state_diff` must reproduce MuJoCo configuration-manifold semantics. Do not add epsilon directly to raw quaternion components.

10 Initial Interpretation Guide

Observation	**Likely interpretation**
CPU black-box FD disagrees with `mjd_transitionFD`	Likely perturbation, state-difference, indexing, or layout bug. Fix before testing Warp.
MJWarp nominal transition already differs strongly	Physics/contact/solver parity issue; derivative comparison is secondary.
MJWarp error decreases then increases as h shrinks	Expected truncation-versus-roundoff tradeoff; choose a stable minimum region.
MJWarp Jacobian changes substantially between identical runs	Nondeterminism/noise may be too large for small h; quantify η/h amplification.
Only contact-changing columns have large error	Likely contact nonsmoothness or contact-set differences; analyze separately.
Stable-contact columns fail while flight columns pass	Investigate solver convergence, collision precision, friction, and active constraints.
One-step derivatives pass but S-step derivatives degrade	Accumulated rollout divergence/roundoff; test larger h, state scaling, or per-substep chaining.
Jacobians differ numerically but Taylor tests and optimizer both work	Exact CPU matrix agreement may not be necessary; optimization utility is the final criterion.

Table 2: Guide for interpreting common validation outcomes.

11 Recommended Order of Implementation

A minimal sequence that maximizes information gained per unit implementation effort is

1. Save a few CPU trajectory fixtures.
2. Implement a manifold-correct black-box CPU central-difference wrapper.
3. Match `mjd_transitionFD` on one-step smooth states.

4. Add stable-contact CPU states and confirm expected behavior.
5. Port exactly the same perturbation layout to MJWarp.
6. Check nominal one-step CPU/MJWarp parity.
7. Run the epsilon sweep on flight and stable-contact fixtures.
8. Add repeated-run tests and contact-set logging.
9. Add the Taylor test.
10. Extend to direct S-substep shooting-map finite differences.
11. Benchmark wall time.
12. Only then integrate the candidate Jacobian source into a small trajectory optimization.

References

[1] MuJoCo Documentation, *API Reference: `mjd_transitionFD`*. https://mujoco.readthedocs.io/en/stable/APIreference/APIfunctions.html#mjd-transitionfd

[2] Google DeepMind, *MuJoCo finite-difference derivative implementation: `engine_derivative_fd.c`*. https://github.com/google-deepmind/mujoco/blob/main/src/engine/engine_derivative_fd.c

[3] MuJoCo Documentation, *MuJoCo Warp Documentation / FAQ*. https://mujoco.readthedocs.io/en/latest/mjwarp/

[4] MuJoCo Documentation, *MuJoCo Warp API*. https://mujoco.readthedocs.io/en/stable/mjwarp/api.html

[5] Google DeepMind, *MuJoCo Warp issue #1546: mesh-contact float32 precision example*, July 21, 2026. https://github.com/google-deepmind/mujoco_warp/issues/1546

[6] MuJoCo Documentation, *API Types: `mjtNum` and `mjUSESINGLE`*. https://mujoco.readthedocs.io/en/stable/APIreference/APItypes.html#mjtNum